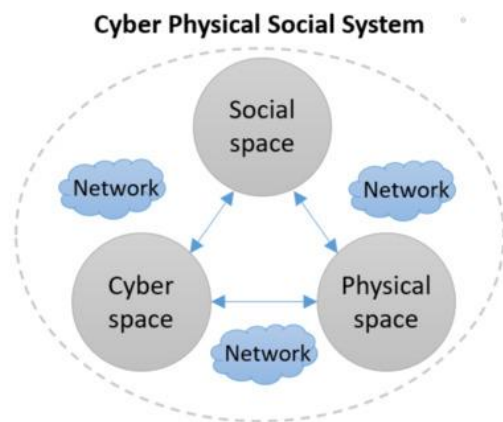
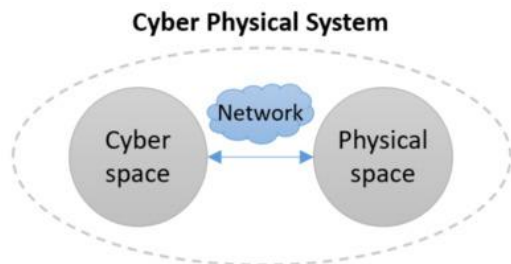


Algorithmic Game Theory

University of Karaj

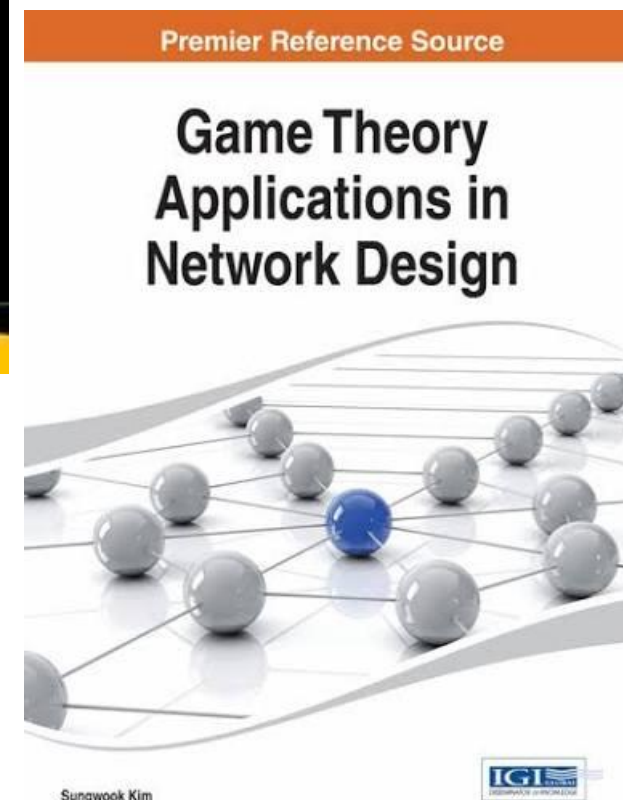
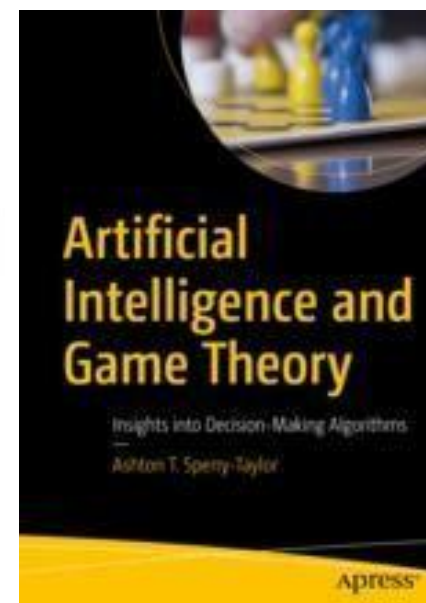
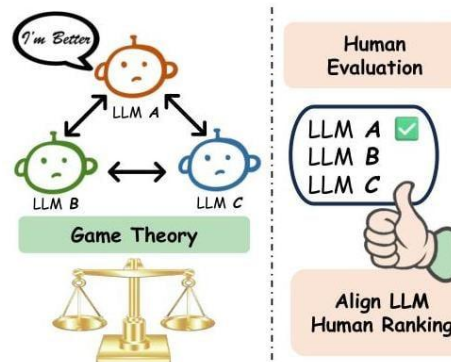
Dr. Amir Noori

Fall-2026



Internet Economics

Understanding Digital and New Media Markets



Course Info

References:

- [Twenty Lectures in Algorithmic Game Theory](#), Cambridge University Press, 2016.
- [Lectures in game theory for computer scientists](#) Cambridge University Press, 2011.

Evaluation:

- Exam 60%
- Exercise 20%
- Seminar 20%

Email: [acnoori \(@\) gmail.com](mailto:acnoori@gmail.com)

Web: acnoori.github.io

Course Materials

- *Twenty Lectures on Algorithmic Game Theory.*
 - T. Roughgarden.
 - Cambridge University Press, 2016.
- *Lectures in game theory for computer scientists.*
 - K. R. Apt and E. Grädel.
 - Cambridge University Press, 2011.
- *Multiagent systems: Algorithmic, game-theoretic, and logical foundations.*
 - Y. Shoham and K. Leyton-Brown.
 - Cambridge University Press, 2008.
- *Algorithmic game theory.*
 - N. Nisan, T. Roughgarden, E. Tardos, and V. V. Vazirani.
 - Cambridge University Press, 2007.

Algorithmic Game Theory

- **Algorithmic game theory** is an area that joins computer science and game theory to study **how smart, selfish groups** make **choices** and **share resources**.



Noam Nisan



Tim Roughgarden,
(Stanford CS364A)



Erich Grädel
(RWTH, Germany)



Kevin Leyton-
Brown (UBC)



MohammadTaghi Hajiaghayi
(Maryland)



Vahab Mirrokni
(Google)



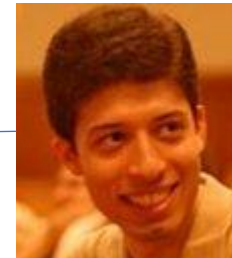
Eva Tardos,
(Cornell)



Vijay Vazirani
(U of California, Irvine)



Yoav Shoham
(Stanford)

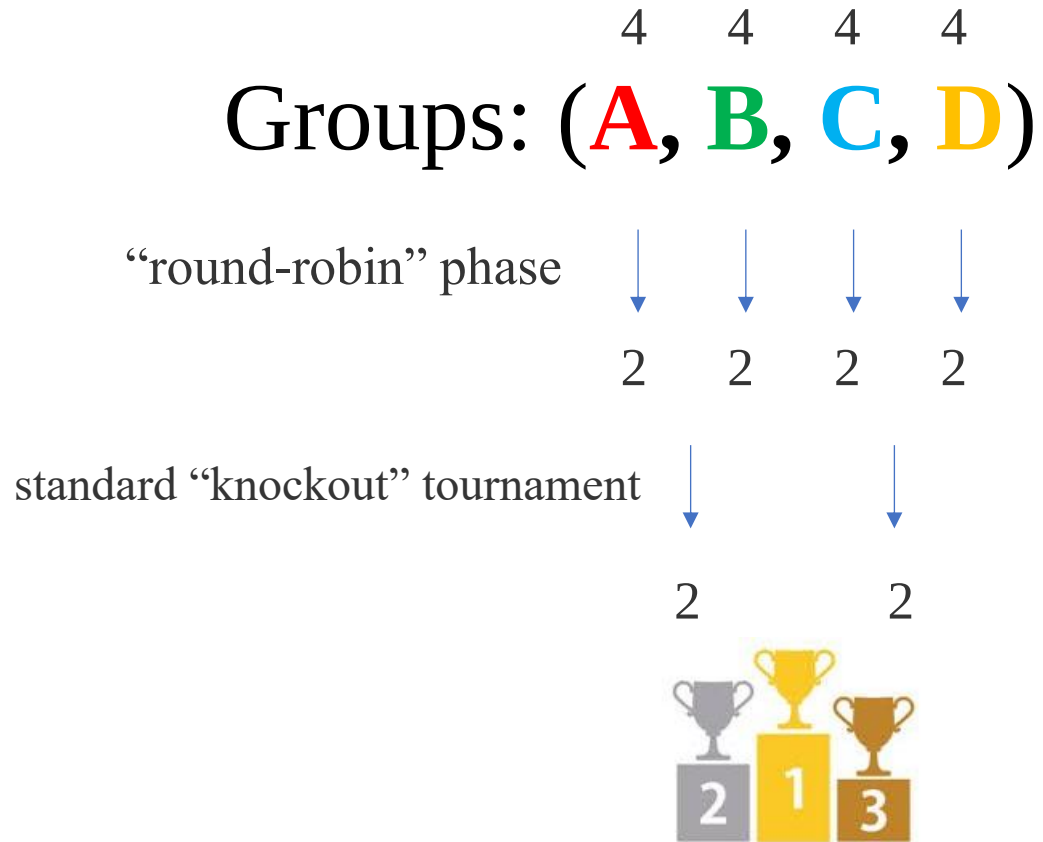


Mohammad Hossein Bateni
(Google)

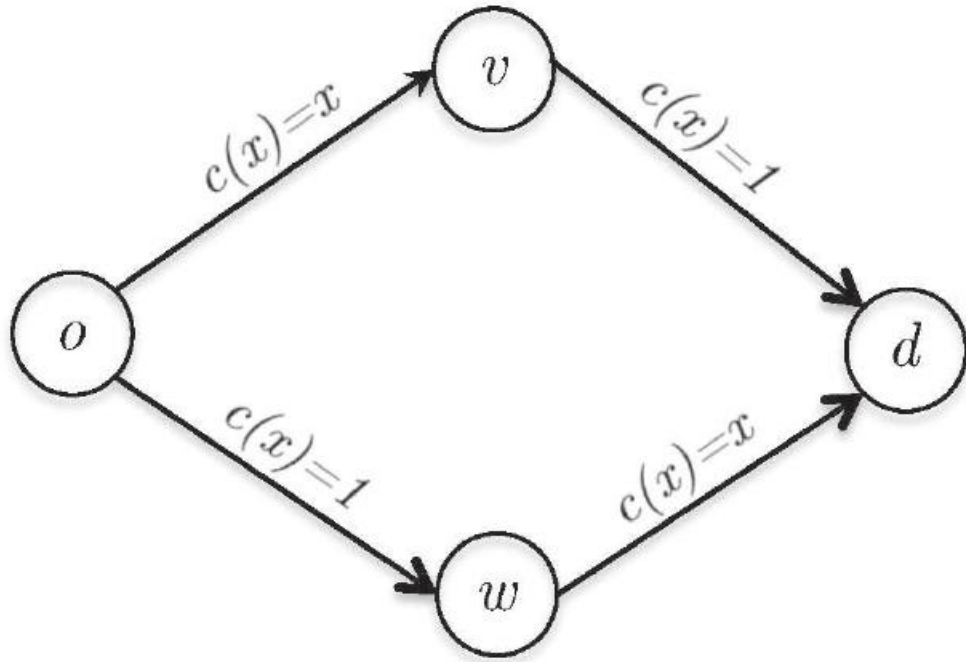
- Market design
- Mechanism Design
- Network Design
- Social Networks
- Scalable Algorithms
- Electronic Commerce
- Internet Economics
- Auctions

How Graphs Can Help You Stay Informed in an Evolving World, ACM26

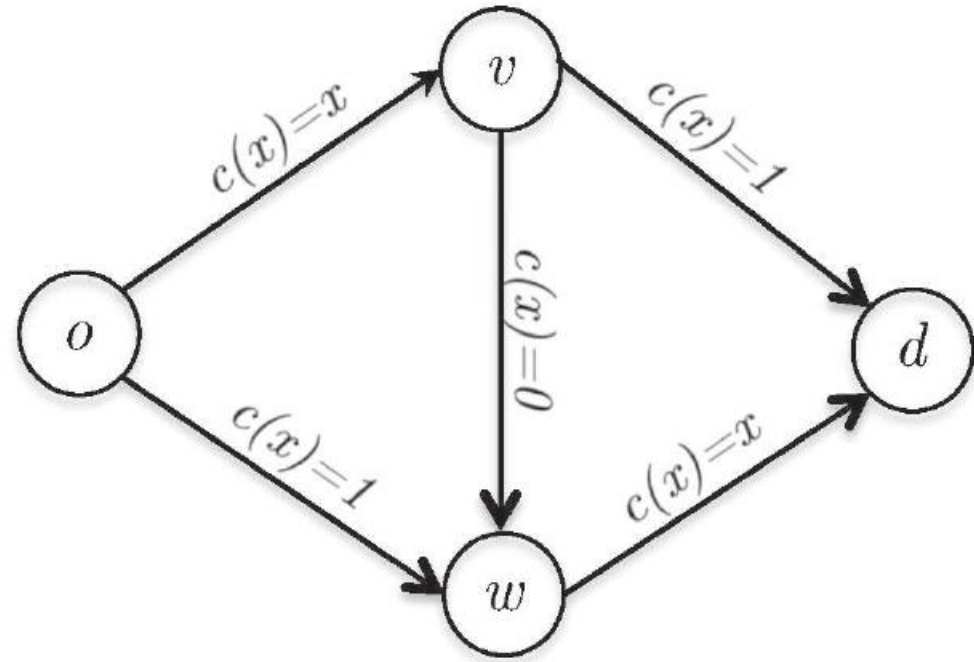
Example (London Olympic 2012: a failed tournament design)



Example (Networks)

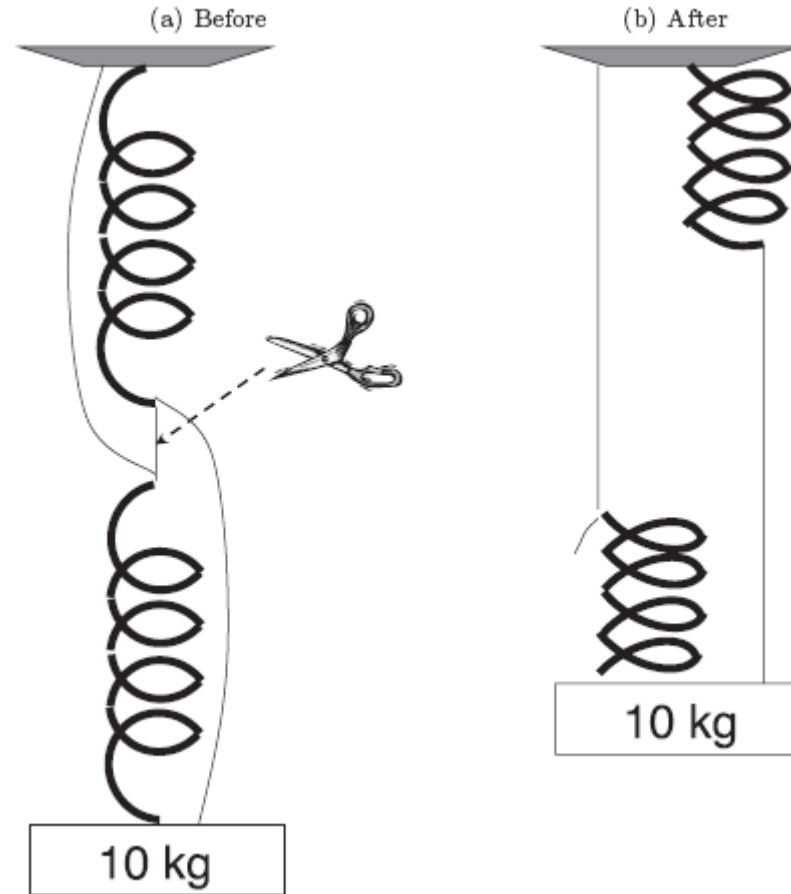


(a) Initial network



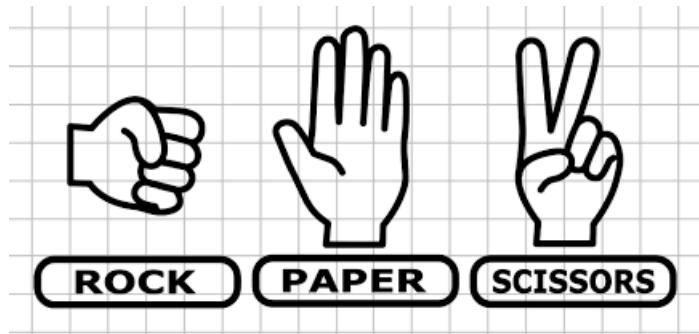
(b) Augmented network

Example (Physics)



Example (Zero-sum Games)

Zero-Sum Games



	Rock	Paper	Scissors
Rock	0, 0	-1, 1	1, -1
Paper	1, -1	0, 0	-1, 1
Scissors	-1, 1	1, -1	0, 0

Example (Supply and Demand)



e.g. Electronics



Strategic Rule-Making (The Consequences of Selfish Behavior)

- **Mechanism Design**: Intersection of game theory, economics, and computer science to explain how rules influence human behavior.
- **Braess's paradox**, demonstrating how adding resources to a network can unintentionally worsen performance when participants act in their own interest.
- **Price of Anarchy** as the gap between selfish outcomes and optimal system performance.
- **Nash equilibrium** and the significant computational challenges involved in predicting stable states in complex games.

Core Concepts

- **Games**: Situations where many independent players make choices that affect everyone else.
- **Equilibrium**: A stable state where no player wants to change their choice because they cannot do any better.
- **Mechanism Design**: Creating rules or algorithms so people are honest even when they want to win.
- **Price of Anarchy**: A number that shows how much worse a system works when everyone is selfish instead of helpful.

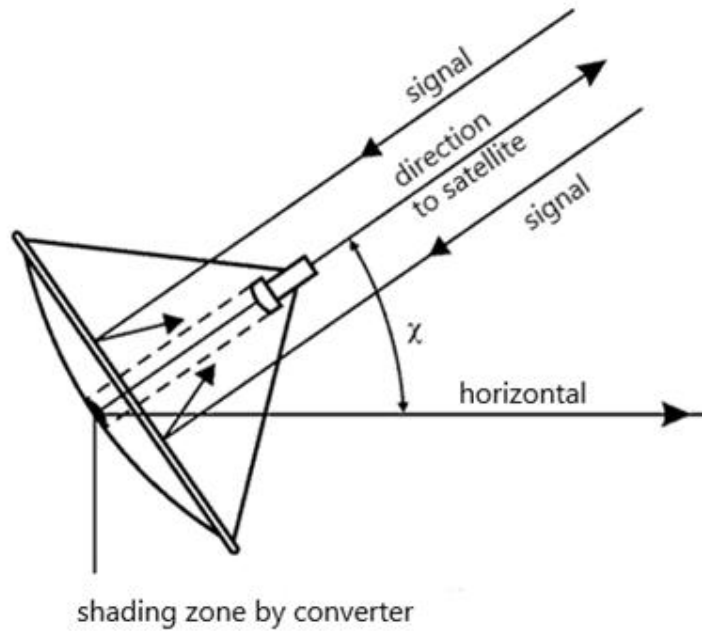
Main Uses

- **Auctions**: Setting up online sales like Google ads or radio frequencies so people bid real prices.
- **Routing**: Helping data move across the internet without getting stuck in jammed paths.
- **Resource Sharing**: Splitting fair goods or tasks among many users.

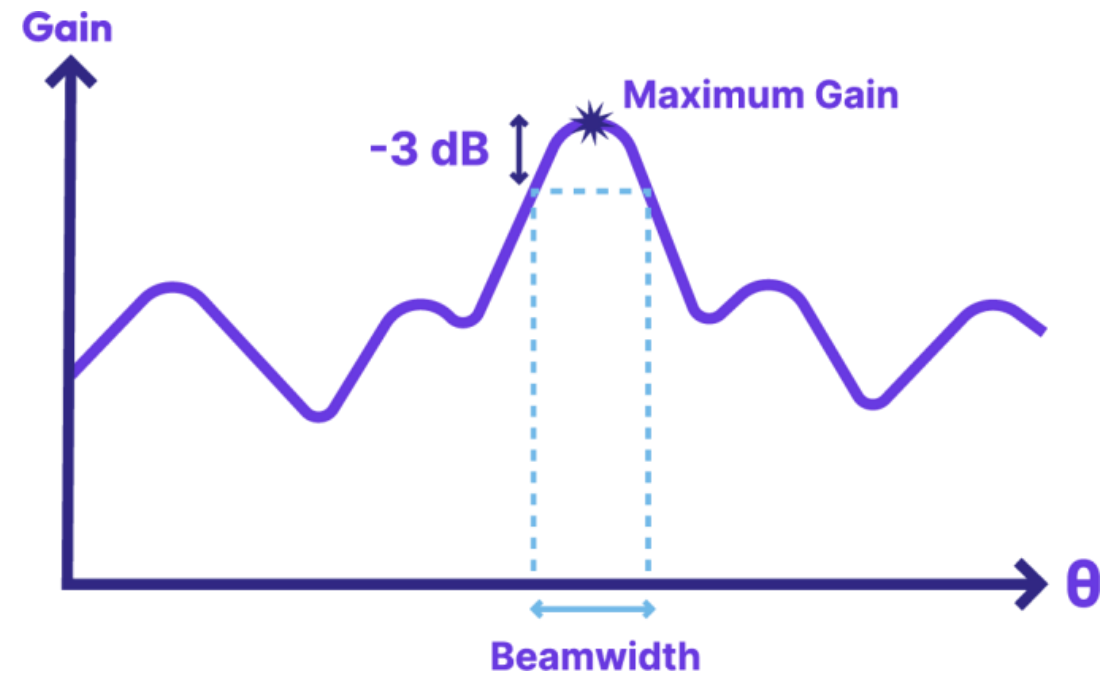
Section 0

Preliminaries: Optimization

Preliminaries (optimization)

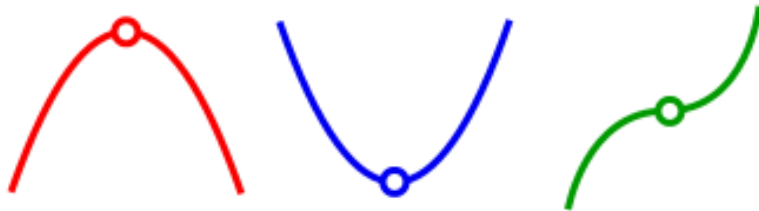


e.g. Antenna Gain



Preliminaries (Solution Concepts)

Extremum points



Maximum

Minimum

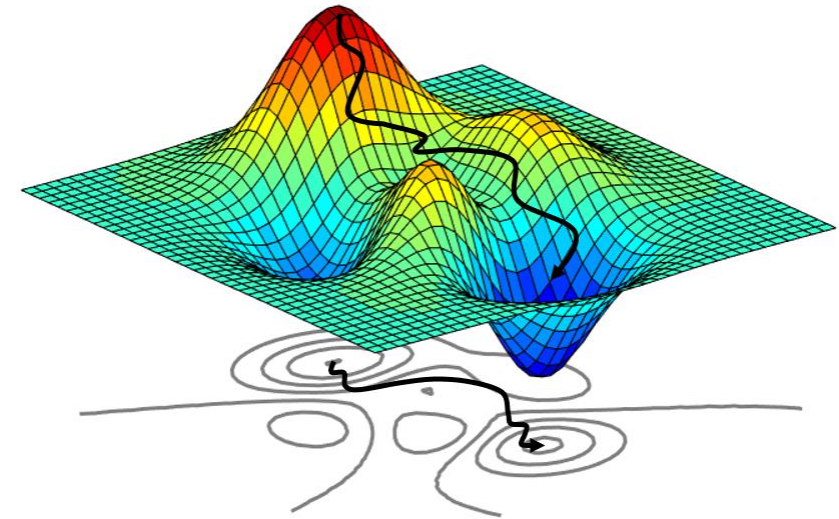
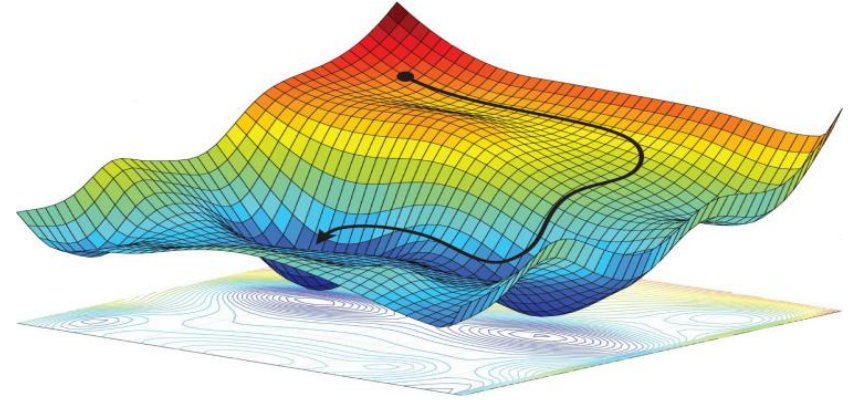
Inflection



Corner



Discontinuity



Preliminaries (Problem Formulation)

Objective Function تابع هدف

Constraints قيود

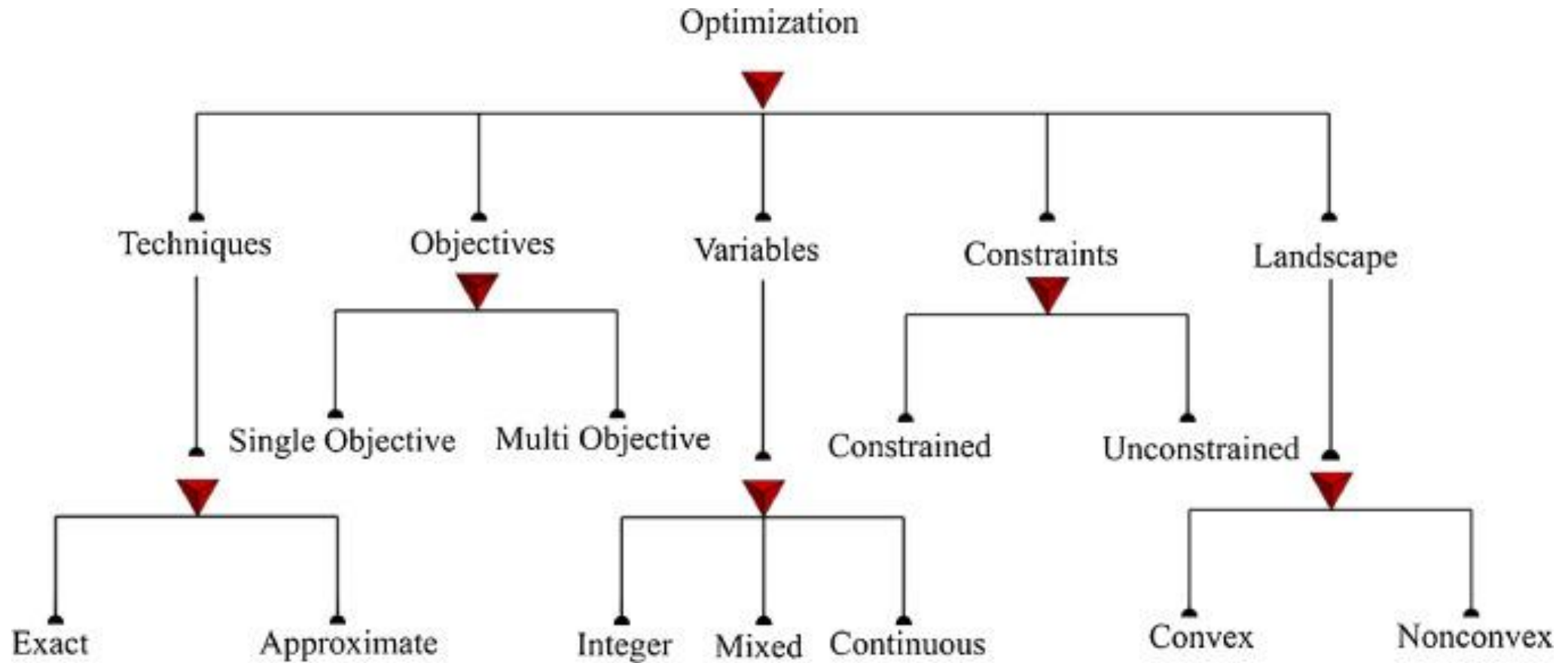
Inequality

Equality

Set

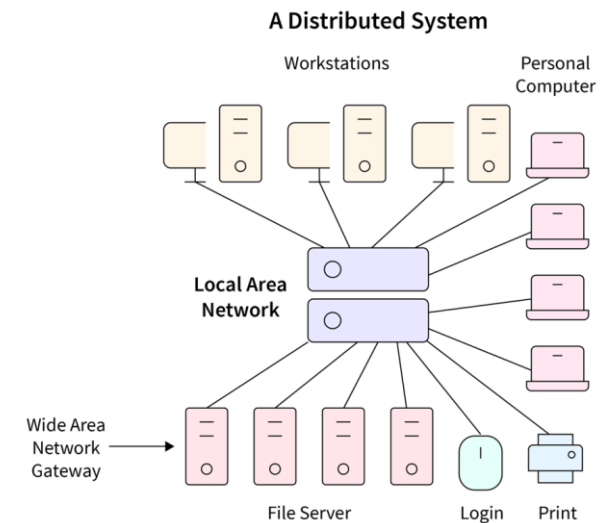
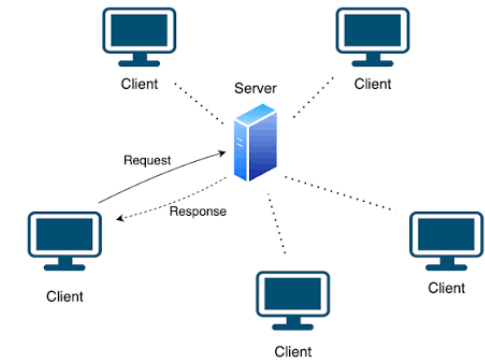
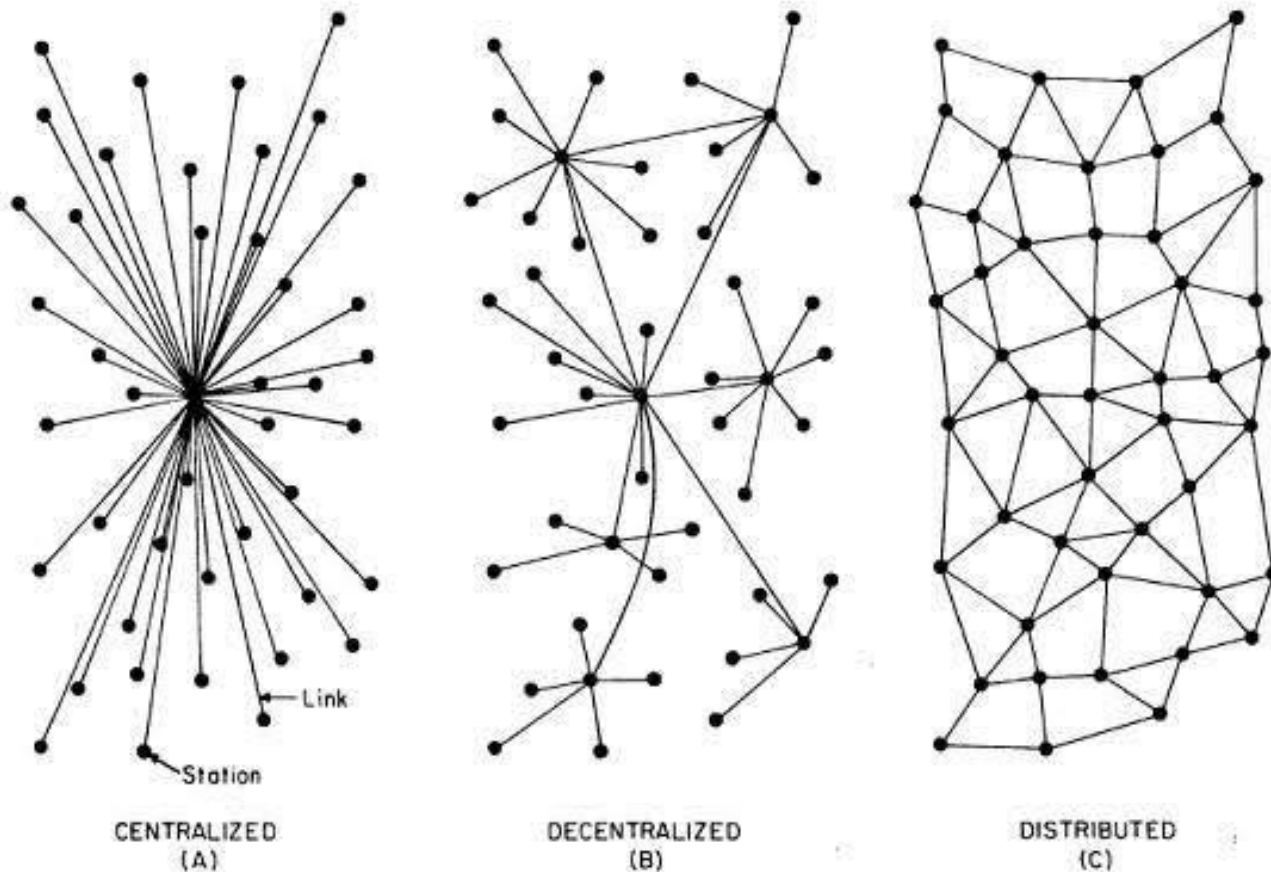
$$\begin{aligned} & \min_x f(x) \\ & \text{subject to} \quad g_i(x) \leq 0, \quad i = 1, \dots, m \\ & \quad \quad \quad h_j(x) = 0, \quad j = 1, \dots, p \\ & \quad \quad \quad x \in X \end{aligned}$$

Preliminaries (Problem Formulation)



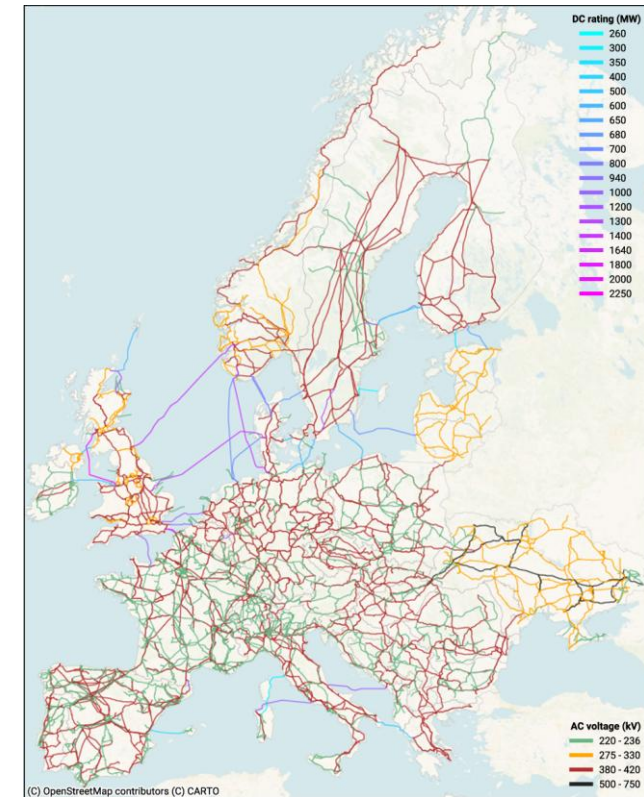
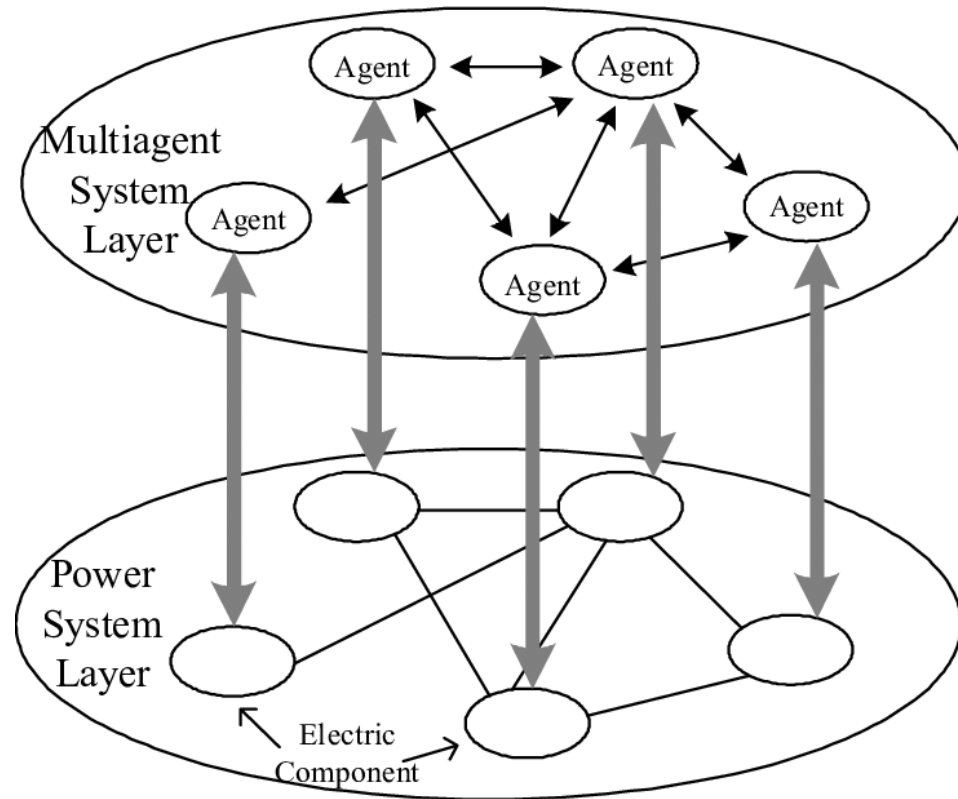
Preliminaries (Multiple Objectives; Spatial and Computational)

e.g. Network Topology

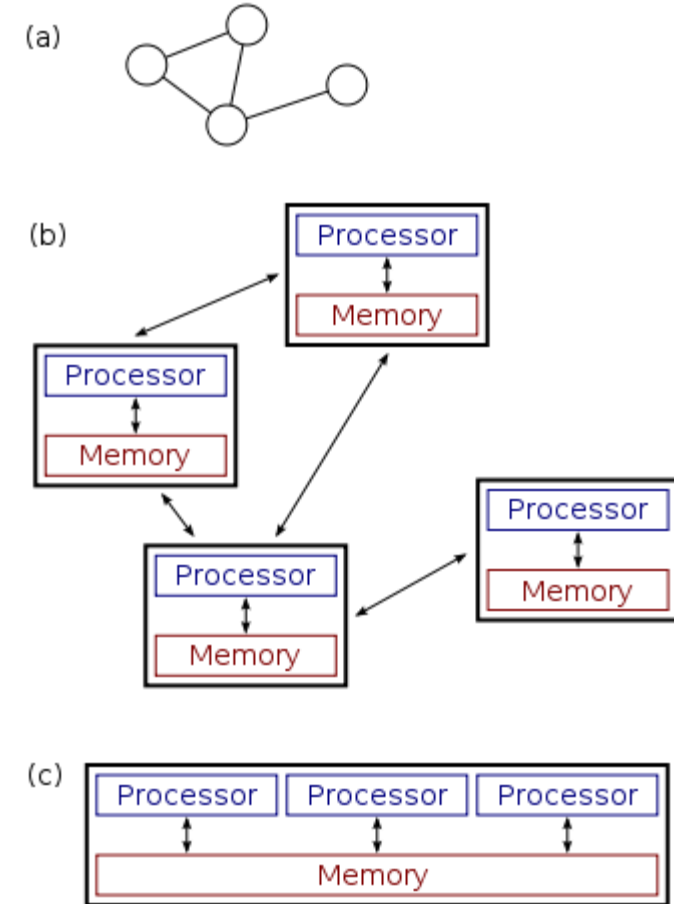
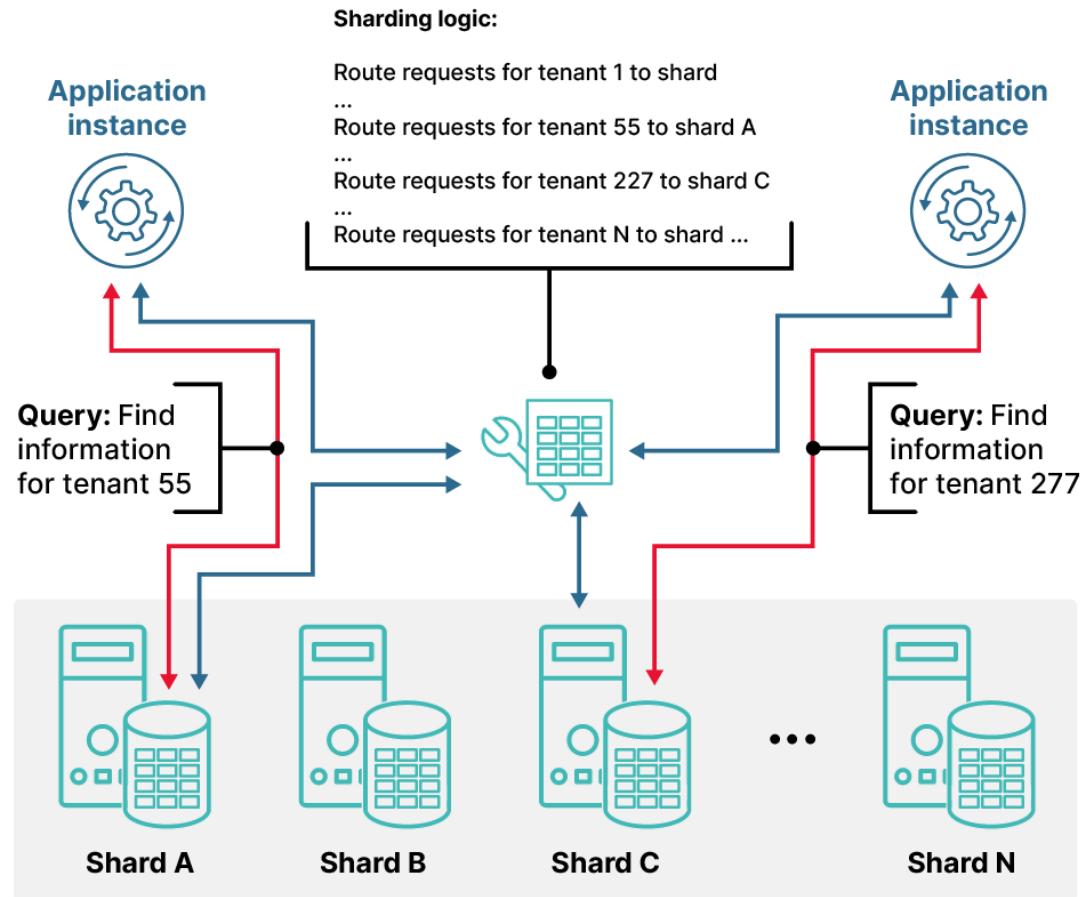


Preliminaries (Multi-agent Systems, Cyber-physical Systems)

e.g. European Power grid (transmission)



Preliminaries (Distributed Systems)



Preliminaries (Some exercises)

Exercise 1 (Constrained Optimization) Formulate some practical problems and computationally solve (python or MATLAB code) in the form of:

- Linear Programming (LP)
- Quadratic Programming (QP)
- Mixed-Integer Linear Programming (MILP)

Preliminaries (Some exercises)

Example 1 (a two-player zero-sum game) Finding the optimal mixed strategy (Nash equilibrium) for a two-player zero-sum game. This is the classic matrix game LP:

$$\max_x \min_j \sum_i x_i A_{ij} \quad \text{s.t.} \quad \sum_i x_i = 1, x_i \geq 0$$

$$\max v \quad \text{s.t.} \quad \sum_i x_i A_{ij} \geq v \quad \forall j, \quad \sum_i x_i = 1, x_i \geq 0$$

$$\min w \quad \text{s.t.} \quad \sum_j A_{ij} y_j \leq w \quad \forall i, \quad \sum_j y_j = 1, y_j \geq 0$$

$$A = \begin{matrix} & \begin{matrix} y1 & y2 & y3 \end{matrix} \\ \begin{bmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{bmatrix} \end{matrix}$$

Preliminaries (Some exercises)

Example 1 (a two-player zero-sum game) Finding the optimal mixed strategy (Nash equilibrium) for a two-player zero-sum game. This is the classic matrix game LP:

```
import numpy as np
from scipy.optimize import linprog

# Payoff matrix: rows = Rock, Paper, Scissors; cols = same
A = np.array([[ 0, -1,  1],
              [ 1,  0, -1],
              [-1,  1,  0]])

n = A.shape[0]

# Maximize v  <=>  minimize -v
# Variables: [x1, x2, x3, v]
c = np.array([0, 0, 0, -1])

# Constraints: for each column j: sum_i x_i A_ij >= v
#              => -sum_i x_i A_ij + v <= 0
A_ub = np.hstack([-A.T, np.ones((n, 1))])
b_ub = np.zeros(n)

# Equality: sum(x) = 1
A_eq = np.array([[1, 1, 1, 0]])
b_eq = np.array([1.0])

bounds = [(0, 1)] * n + [(None, None)]

res = linprog(c, A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
              bounds=bounds, method='highs')

print("Row strategy x:", res.x[:n])
print("Game value v  :", res.x[n])
```

```
A = [ 0 -1  1;
      1  0 -1;
     -1  1  0];
n = size(A,1);

% Variables: [x1 x2 x3 v]
f = [0 0 0 -1];

% -A' * x + v <= 0
A_ub = [-A', ones(n,1)];
b_ub = zeros(n,1);

% sum(x) = 1
Aeq = [1 1 1 0];
beq = 1;

lb = [0 0 0 -inf];
ub = [1 1 1  inf];

[x, fval] = linprog(f, A_ub, b_ub, Aeq, beq, lb, ub);

disp('Row strategy:'), disp(x(1:n))
disp('Game value  :'), disp(-fval) % since we minimize
```

```
*** Row strategy x: [0.33333333 0.33333333 0.33333333]
Game value v  : -0.0
```

Section 1

Game Theory

Introduction and Examples

Games against nature (decision theory for a single agent)

Expected utility theory for a single agent is sometimes called the theory of "games against nature". Consider this example.

Example 1: Planning a party: Our agent is planning a party, and is worried about whether it will rain or not. The utilities and probabilities for each state and action can be represented as follows:

Normal or Matrix from		Nature's states	
		Rain ($p=1/3$)	No rain ($\sim p=2/3$)
Party planner's possible actions	Outside	1	3
	Inside	2	2

https://web.stanford.edu/class/symsys202/Game_Theory_Through_Examples.html

Expected Utility

The expected utility of an action A given uncertainty about a state S

$$EU(A|S) = \text{Prob}(S|A) * \text{Utility}(S|A) + \text{Prob}(\text{not } S|A) * \text{Utility}(\text{not } S|A)$$

(Prob. = Probability)

- Note that action A can be viewed as a compound gamble or outcome.
- Also, note that the probability of a state can depend on the agent's choice of action, although, in the above example, it does not.

Expected Utility

$$EU(A) = \text{Prob}(S|A) * \text{Utility}(S|A) + \text{Prob}(\text{not } S|A) * \text{Utility}(\text{not } S|A)$$

For the party problem:

$$EU(\text{Outside}) = (1/3)(1) + (2/3)(3) = 2.33;$$

$$EU(\text{Inside}) = (1/3)(2) + (2/3)(2) = 2$$

Therefore, choose Outside, the action with the higher expected utility.

(Noncooperative) game theory (decision theory for more than one agent, each acting autonomously (no binding agreements))

In the examples below, we'll assume two self-utility maximizing agents (or players), each of whom has complete information about the options available to themselves and the other player as well as their own and the other's payoffs (utilities) under each option.

Example 2: Friends hoping to see each other

Consider two people, Chris and Kim. They both enjoy each other's company, but neither can communicate with the other before deciding whether to stay at home (where they would not see each other) or go to the beach this afternoon (where they could see each other). Each prefers going to the beach to being at home, and prefers being with the other person rather than being apart. This game can be represented by the following normal (or matrix) form:

Normal or Matrix form		Kim	
		Home	Beach
Chris	Home	(0,0)	(0,1)
	Beach	(1,0)	(2,2)

Strategy, Outcome, Payoff

Each player has a set of **strategies** ($=\{ \text{Home, Beach} \}$ for both players in this example).

Specifying one strategy i for the row player (Chris) and
one strategy j for the column player (Kim)

yields an **outcome**, which is represented as a pair of **payoffs** (R_{ij}, C_{ij}) ,
where R_{ij} is the utility the row player receives,
and C_{ij} is the utility the column player receives.

Dominant Strategy

In this example, going to the beach is a (strictly) **dominant strategy** for each player, because it always yields the best outcome, no matter what the other player does. Thus, if the players are both maximizing their individual expected utilities, each will go to the beach. So Beach-Beach is a dominant strategy equilibrium for this game. Because of this, Kim and Chris, if they are rational, do not need to **cooperate** (make an agreement) ahead of time. Each can just pursue their own interest, and the best outcome will occur for both.

(Noncooperative) game theory (decision theory for more than one agent, each acting autonomously (no binding agreements))

Example 3 - "Friends" with asymmetric preferences

Now consider Betty and John. John likes Betty, but Betty doesn't like John that much. Each knows this, and neither wants to call the other before deciding what to do this afternoon: stay at their respective homes or go to the neighborhood swimming pool. Here is the normal form:

Normal or Matrix form		John	
		Home	Pool
Betty	Home	(2,0)	(2,1)
	Pool	(3,0)	(1,2)

Dominant Strategy

In this case, Betty's best strategy depends on what John does. But if she assumes John is rational, she will reason that he will not stay home, because going to the pool is a dominant strategy for him. Knowing this, she can decide to stay home (because $2 > 1$). This is called iterated dominance. In this example, Betty gets higher utility than John because of their relative preferences, and John gets less utility than he would have if Betty wanted to be with him.

Pareto Optimal (Efficient)

In this example, Pool-Home (3,0), Home-Pool (2,1), and Pool-Pool (1,2) are all **Pareto optimal** outcomes. An outcome is **Pareto optimal** (or **efficient**) if no agent can be made better off than that outcome without making another agent worse off. The equilibrium outcomes in both this example and the previous one are Pareto optimal.

(Noncooperative) game theory (decision theory for more than one agent, each acting autonomously (no binding agreements))

Example 4 - Prisoners' dilemma

Consider Stan and Leland, two prisoners who have each been offered a deal to turn state's witness (defect) against the other. They can't communicate. They had originally agreed to remain in solidarity, i.e. not testify against each other, but since the agreement cannot be enforced, each must choose whether to honor it. If both remain in solidarity, then they will each only be convicted of a minor charge. If only one defects, then the state will throw the book at the other and let the defector go. If they both defect, each will get convicted of a serious charge. The payoff matrix (higher positive utility implies a better outcome) is as follows:

Normal or Matrix from		Leland	
		Solidarity	Defection
Stan	Solidarity	(3,3)	(1,4)
	Defection	(4,1)	(1,1)

Dominant Strategy (weakly, strictly)

In this game, the strategy of defection is **weakly dominant** for each player, meaning that whatever the other player does, defecting yields an outcome at least as good and possibly better than remaining in solidarity would. Note that if the bottom right cell payoffs were (2,2) instead of (1,1), then defecting would be **strictly dominant** for each player. Either way, Defection-Defection is a **dominant strategy** equilibrium. However, it is not **Pareto optimal**. Both players could be made better off if neither defected against the other.

Social Dilemma

This is an example of a **social dilemma**: a situation in which each agent's autonomous maximization of self-utility leads to an inefficient outcome. Such a situation can occur for any number of people, not just two. An agreement by two people to trade with each other (involving goods, services, and/or money) set's up a prisoners' dilemma-type game whenever the agreement cannot be enforced.

Coordination

Example 5 - Coordination

Let's go back to Chris and Kim. They are going to the same conference, and each is expecting the other to be there, but they haven't seen each other yet. The conferees have their choice of two activities on the first afternoon: swimming or hiking. They both hope to see each other -- if they don't they will have no fun, and each prefers swimming over hiking. They must each decide what to do before knowing where the other is going. Here is the normal form:

Normal or Matrix from		Kim	
		Swim	Hike
Chris	Swim	(2,2)	(0,0)
	Hike	(0,0)	(1,1)

Nash Equilibrium

The best outcome is obviously Swim-Swim, but going swimming is not dominant for either player. Both Swim-Swim and Hike-Hike have the property that each player's strategy is the best (or tied for the best) response to the other player's strategy in that pairing. This defines a more general equilibrium notion called the **Nash equilibrium**. The dominance equilibria of examples 1-3 are all Nash equilibria as well.

Mixed strategy

A third equilibrium exists in this game involving what are called **mixed strategies**. A **mixed strategy** is a probability distribution over the pure strategies (which are Swim and Hike for each player in this example). (Note that the players do not have to have the same set of strategies available to them, even though that has been the case in all our examples.)

In this example, if each player individually throws a die and goes swimming if the die comes up 1 or 2, and goes hiking if the die comes up 3, 4, 5, or 6, the resulting expected utility ($2/3$ for each player) cannot be improved upon for either player given that the other player uses this strategy.

Mixed strategy

In 1950, **John Nash** (depicted somewhat fictitiously in the film *A Beautiful Mind* -- the book is more accurate!) proved that every **finite game**, involving any number of players, has **at least one (Nash) equilibrium**, though there might not be any that involve only pure strategies for all players. In this example, there are three equilibria: the mixed strategy equilibrium (Swim, $1/3$; Hike, $2/3$)-(Swim, $1/3$; Hike, $2/3$), and two pure strategy equilibria -- Swim-Swim and Hike-Hike.

Theorem 1.1 (Nash's Theorem) Every finite two-player game has a Nash equilibrium.

Equilibrium Selection

When there is more than one equilibrium, and players cannot make binding agreements, they must try to coordinate to arrive at an equilibrium outcome. When only one equilibrium is also Pareto optimal, as Swim-Swim is in this case, that fact should suggest to rational players that it will be the one around which they coordinate. Many other criteria for equilibrium selection have been studied (e.g. **focal points**, **subgame perfection**, **stability** -- see the reading on game theory).

Coordination

Example 6 - "Battle of the sexes"

Finally, let's consider Roy and Jen. They are going to the same conference as Kim and Chris in example 5. They each would prefer to be in the same place (the swim or the hike), but their preferences differ about which it should be. Roy would rather go swimming, and Jen would rather go hiking. Here is the matrix form:

Normal or Matrix form		Jen	
		Swim	Hike
Roy	Swim	(3,2)	(1,1)
	Hike	(1,1)	(2,3)

Nash Equilibrium

This game has three Nash equilibria: Swim-Swim, Hike-Hike, and $(\text{Swim}, 2/3; \text{Hike}, 1/3) - (\text{Swim}, 1/3; \text{Hike}, 2/3)$.

Note that the mixed strategies differ for each player in the third equilibrium: each goes to their preferred activity with $2/3$ probability.

Focal Point

All of the equilibria are Pareto optimal this time, so that does not help for selection. Only the mixed strategy equilibrium results in equal expected utilities for the two players, so if both value equality or symmetry, this might be the **focal point**. But of course it will be difficult for Roy and Jen to see that unless **they have studied game theory!**

Focal Point

All of the equilibria are Pareto optimal this time, so that does not help for selection. Only the mixed strategy equilibrium results in equal expected utilities for the two players, so if both value equality or symmetry, this might be the **focal point**. But of course it will be difficult for Roy and Jen to see that unless **they have studied game theory!**

Zero-Sum Games and Minimax Theorem

Imagine you are playing a game against a *super-smart* opponent. You want to find the move that gives you the highest possible reward (*greedily*). But you know your opponent is also smart, and they will try to block your best move and force you into a bad position.

If you play greedily, aiming for the best possible outcome, a smart opponent will trap you, and you will end up with the *worst possible outcome*.

Zero-Sum Games and Minimax Theorem

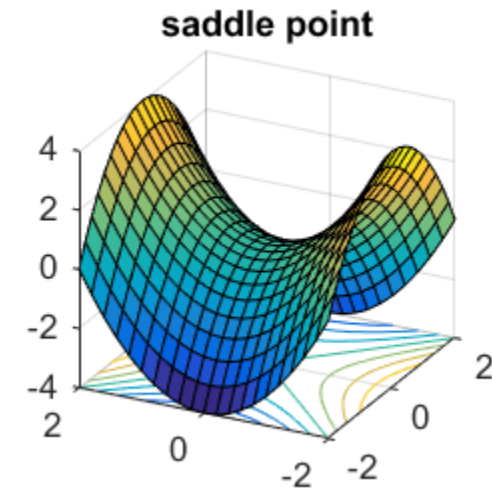
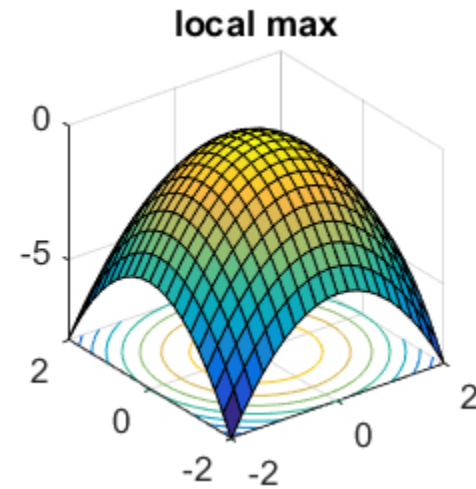
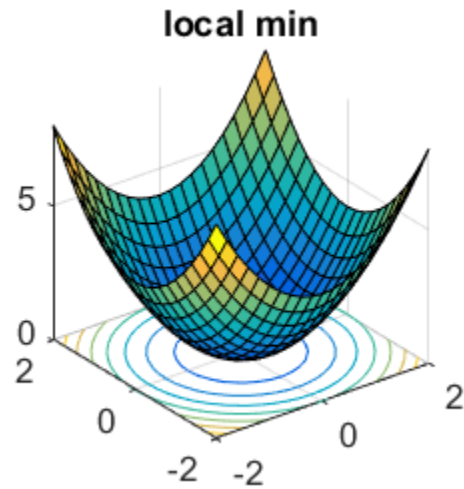
The Solution: “*Maximize your Minimum*”

Von Neumann (1928) realized that in a zero-sum game, you shouldn't look for the best-case scenario. You should look for the best of the worst-case scenarios.

- *Minimax* means: You want to minimize your maximum possible loss.
- (Alternatively called *Maximin*): You want to maximize your minimum possible gain.

Zero-Sum Games and Minimax Theorem

Saddle-point dynamics:

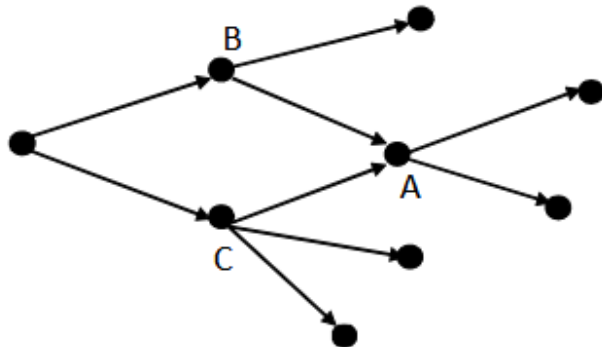


Game Tree

Game Tree: A tree is a set of nodes and directed edges connecting these nodes such that

- 1. there is an initial node, for which there is no incoming edge;
- 2. for every other node, there is exactly one incoming edge;
- 3. for any two nodes, there is a unique path that connect these two nodes.

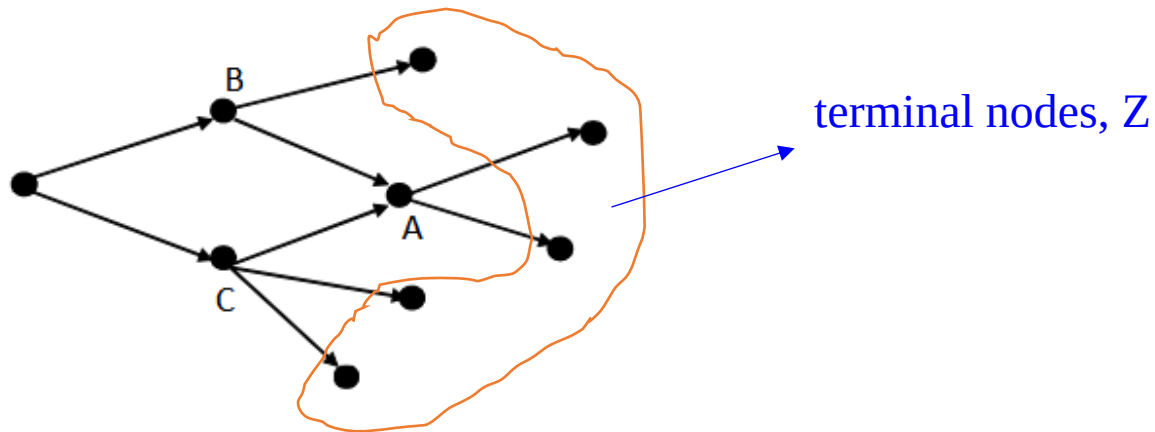
The nodes that are not followed by another node are called terminal



Games in Extensive-form

Definition (*Extensive form*) A Game consist of

- a set of players, $N = \{1, 2, \dots, n\}$, generic players $i, j \in N$
- a tree,
- an allocation of non-terminal nodes of the tree to the players,
- an *informational partition* of the non-terminal nodes, and
- payoffs for each player at each terminal node.



Outcome and Payoffs
(At each terminal node)

$$u_i : Z \rightarrow \mathbb{R}$$

Games in Extensive-form

Example) Matching Pennies with Perfect Information

These payoffs are von Neumann-Morgenstern utilities. That is, each player tries to maximize the expected value of his own payoffs given his beliefs about how the other players will play the game.

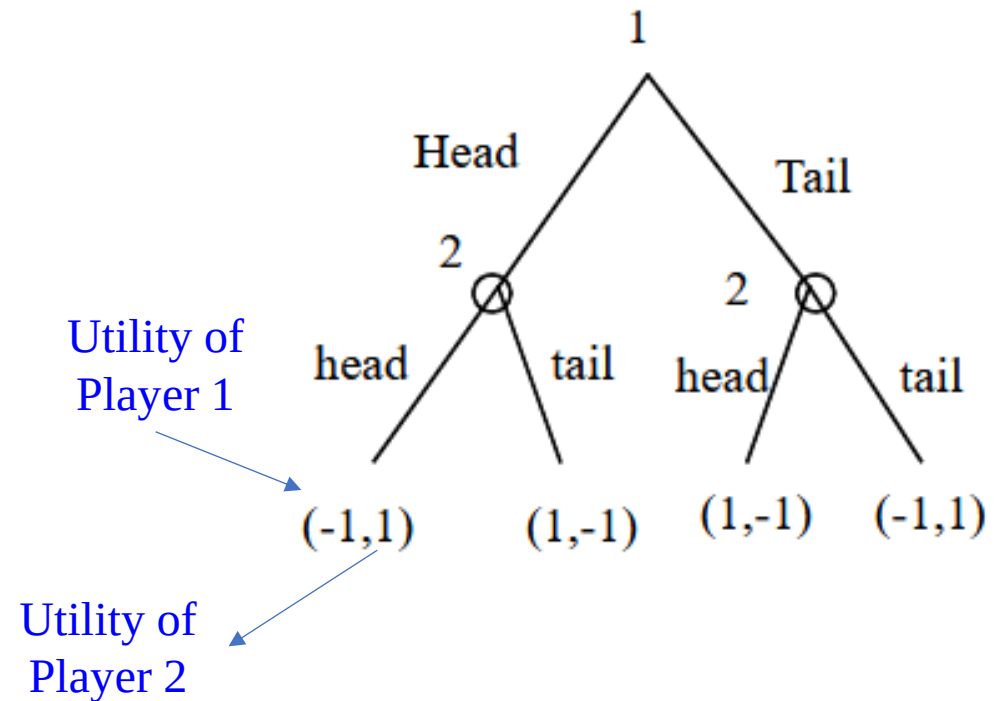


Figure 3.4: Matching Pennies with Perfect Information

Information Set

Definition An *information set* is a collection of nodes such that

1. the **same player i** is to move at each of these nodes;
2. the **same moves** are available at each of these nodes.

Definition An *information partition* is an allocation of each non-terminal node of the tree to an information set; the starting node must be “alone”.

A game is said to have *perfect information* if every information set has only one element.

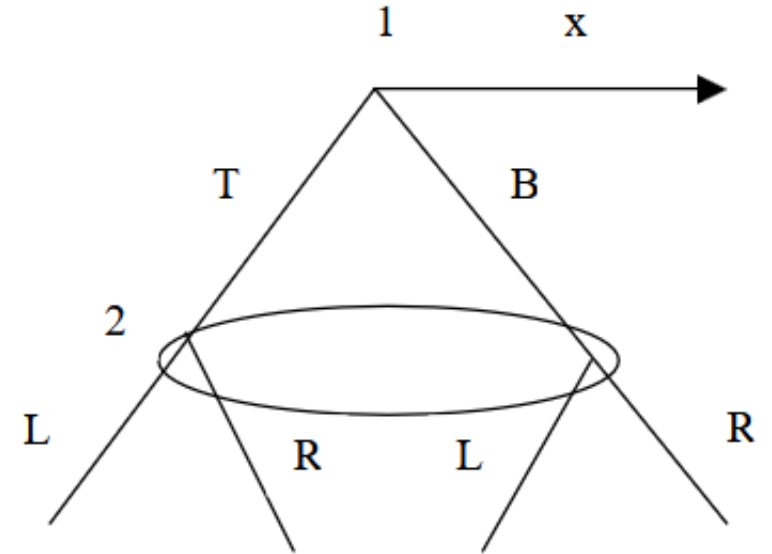


Figure 3.5: A game

Perfect Information

Definition A game is said to have *perfect information* if every information set has only one element.

- Recall that in a tree, each node is reached through a unique path. Hence, in a perfect-information game, a player can construct the previous play perfectly.
- For instance in Figure, Player 2 knows whether Player 1 chose Head or Tail. And Player 1 knows that when he plays Head or Tail, Player 2 will know what Player 1 has played.

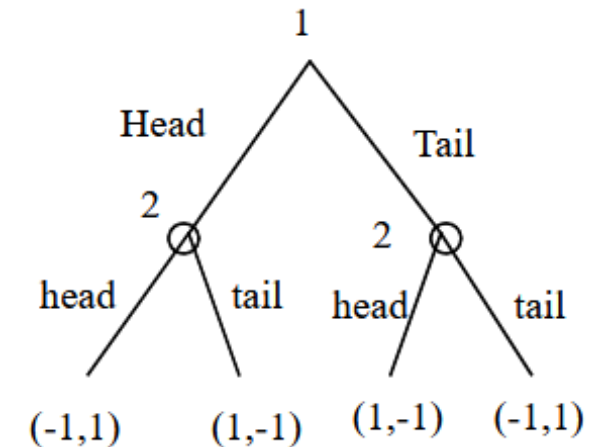
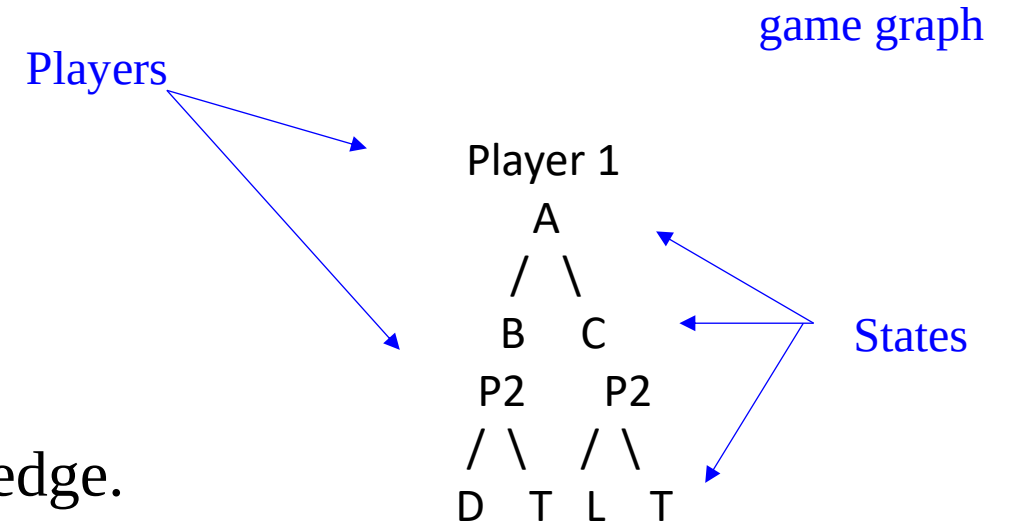


Figure 3.4: Matching Pennies with Perfect Information

Game Graphs and Reachability Games

Suppose:

- Player 1 wants to reach T.
- Player 2 wants to prevent reaching T.
- The token starts at A.
- The owner of a node chooses the outgoing edge.



Can Player 1 force the token to reach T?

- its solution can be explained through backward propagation / attractors.

Actions: {Left, Right}

Strategy: A seq. Of Actions: {LR}

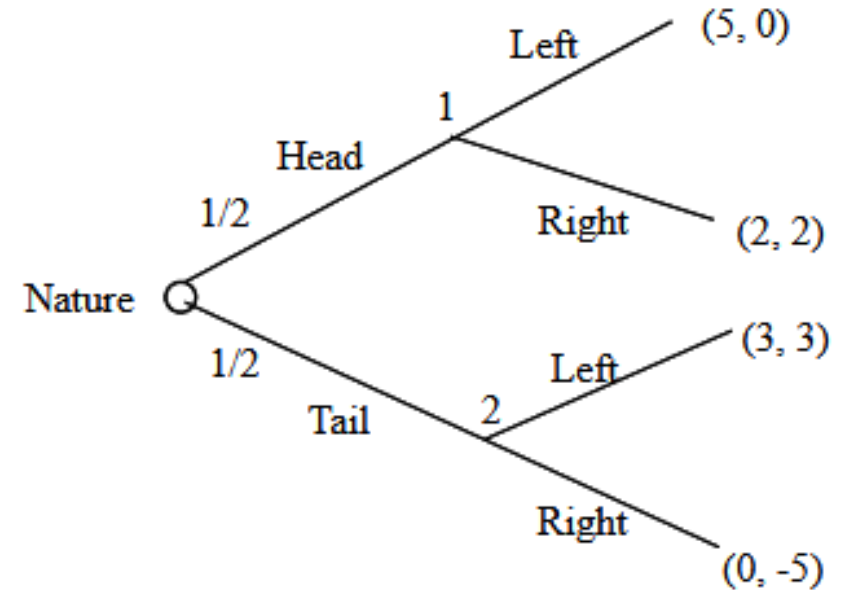
compute the region from which Player 1 can force the target and the complementary region from which Player 2 can prevent it.

Nature as a player

A Game with chance.

Definition A strategy of a player is a complete contingent-plan determining which action he will make at each information set he is to move (including the information sets that will not be reached according to this strategy).

More mathematically, a strategy of a player i is a function s_i that maps every information set h_i of player i to an action that is available at h_i .



Strategy $s_i: h_i \rightarrow a_i$

Strategies

Player 1 has only one information set. Hence, the set of strategies for Player 1 is {Head, Tail}.

On the other hand, Player 2 has two information sets. Hence, a strategy of Player 2 determines what to do at each information set, i.e., depending on what Player 1 does.

So, her strategies are:

HH = Head if Player 1 plays Head, and Head if Player 1 plays Tail

HT = Head if Player 1 plays Head, and Tail if Player 1 plays Tail;

TH = Tail if Player 1 plays Head, and Head if Player 1 plays Tail;

TT = Tail if Player 1 plays Head, and Tail if Player 1 plays Tail.

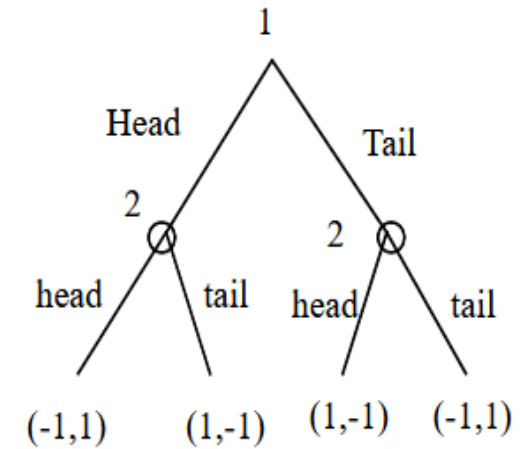
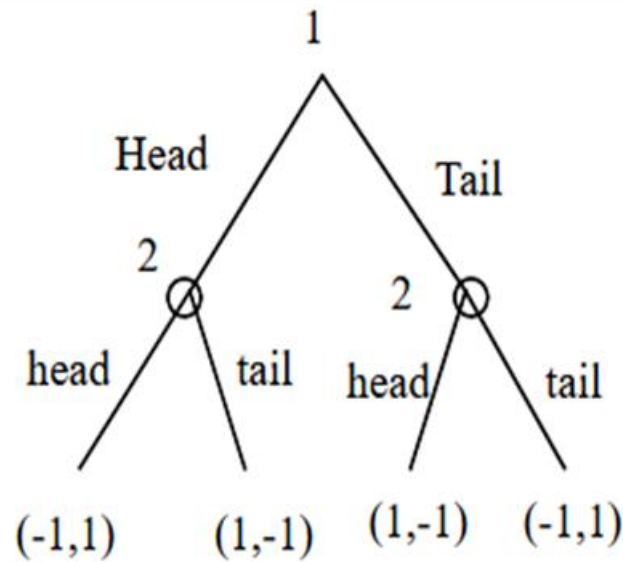
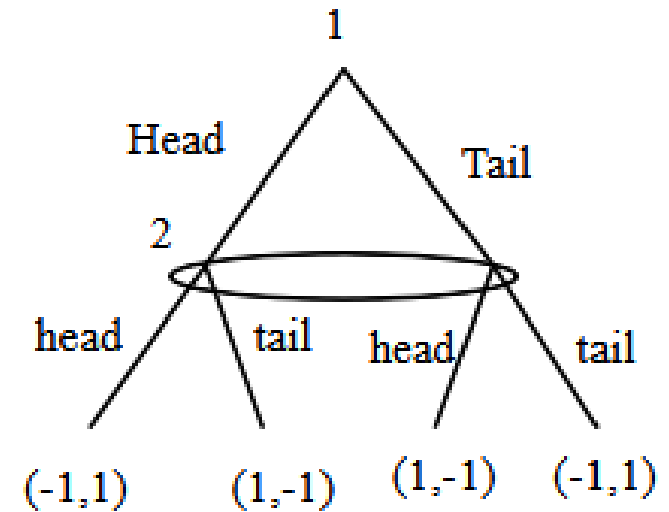


Figure 3.4: Matching Pennies with Perfect Information

Normal-form vs Extensive-form



	<i>HH</i>	<i>HT</i>	<i>TH</i>	<i>TT</i>
<i>Head</i>	$-1, 1$	$-1, 1$	$1, -1$	$1, -1$
<i>Tail</i>	$1, -1$	$-1, 1$	$1, -1$	$-1, 1$



<i>1 \ 2</i>	<i>head</i>	<i>tail</i>
<i>Head</i>	$-1, 1$	$1, -1$
<i>Tail</i>	$1, -1$	$-1, 1$

Types of Game (Summary)

1. Zero-sum vs. non-zero-sum games
2. Symmetric vs. asymmetric games (Sym: equal rules, strategic choices)
3. Simultaneous move vs. sequential move games
4. Cooperative vs. non-cooperative games
5. Perfect vs. imperfect information

(Perfect information: full transparency, every move is visible)

Section 2

Mechanism Design Basics
(the science of rule-making)

The design of single-item auctions

Course Goal 1: Understand how to design systems with strategic participants that have good performance guarantees.

Example: Consider a seller with a single item, such as a slightly antiquated smartphone. This is the setup in a typical eBay auction, for example. There is some number n of (strategic!) bidders who are potentially interested in buying the item.

- Bidder behavior in various auction formats
- model of what a bidder wants

The design of single-item auctions

- Each bidder i
 - a nonnegative valuation v_i
(her maximum willingness-to-pay for the item being sold)
 - Thus bidder i wants to acquire the item as cheaply as possible, provided the selling price is at most v_i .
 - Another important assumption is that this valuation is private, meaning it is unknown to the seller and to the other bidders.
 - quasilinear utility model:
 - If a bidder i loses an auction, her utility is 0.
 - If the bidder wins at a price p , her utility is $v_i - p$.

Sealed-Bid Auctions

- 1) Each bidder i privately communicates a bid b_i to the seller, in a sealed envelope, if you like.
- 2) The seller decides who gets the item (if anyone).
- 3) The seller decides on a selling price.
 - There is a way to implement the 2nd step, **give the item to the highest bidder**.
 - There are multiple reasonable ways to implement the 3rd step, and the choice of implementation significantly affects bidder behavior. For example, suppose we try to be altruistic and charge the winning bidder nothing. This idea backfires badly, with the auction devolving into a game of “who can name the highest number?”

First-Price Auctions

In a first-price auction, the winning bidder **pays her bid**.

First-price auctions are **hard** to reason about.

- First, as a participant, it's hard to figure out **how to bid**.
- Second, as a seller or auction designer, it's hard to predict **what will happen**.

Second-Price Auctions and Dominant Strategies (1)

What happens when you win an eBay auction?

If you bid \$100 and win, do you pay \$100?

Not necessarily: eBay uses a “proxy bidder” that increases your bid on your behalf until your maximum bid is reached, or until you are the highest bidder, whichever comes first.

For example, if the highest other bid is only \$90, then you only pay \$90 (plus a small increment) rather than your maximum bid of \$100. If you win an eBay auction, the sale price is essentially the highest other bid—**the second highest overall.**

Second-Price Auctions and Dominant Strategies (2)

A **second-price** or **Vickrey** auction is a sealed-bid auction in which the highest bidder wins and pays a price equal to the second-highest bid. To state the most important property of second-price auctions, we define a **dominant strategy** as a strategy (i.e., a bid) that is guaranteed to maximize a bidder's utility, no matter what the other bidders do.

Second-Price Auctions and Dominant Strategies (3)

Proposition 2.1 (Incentives in Second-Price Auctions) In a second-price auction, every bidder i has a dominant strategy: set the bid b_i equal to her private valuation v_i .

Proposition 2.1 **implies** that second-price auctions are particularly easy to participate in. When selecting a bid, a bidder doesn't need to reason about the other bidders in any way—how many there are, what their valuations are, whether or not they bid truthfully, etc.

This is completely different from a first-price auction, where it never makes sense to bid one's valuation—this guarantees zero utility—and the optimal amount to underbid depends on the bids of the other bidders.

Second-Price Auctions and Dominant Strategies (4)

Another important property is that a truthful bidder—meaning one that bids her true valuation—never **regrets** participating in a second-price auction.

Proposition 2.2 (Nonnegative Utility) In a second-price auction, every truthful bidder is guaranteed nonnegative utility.

Ideal Auctions (1)

Second-price single-item auctions are “ideal” in that they enjoy **three** quite different and **desirable** properties.

Definition 2.3 (Dominant-Strategy Incentive Compatible) An auction is dominant-strategy incentive compatible (DSIC) if truthful bidding is always a dominant strategy for every bidder and if truthful bidders always obtain nonnegative utility.

(a.k.a. individual rationality or voluntary participation.)

Ideal Auctions (2)

Define the social welfare of an outcome of a single-item auction by

$$\sum_{i=1}^n v_i x_i,$$

sale price does not appear in the definition of the social welfare?

where x_i is 1 if i wins and 0 if i loses. Because there is only one item, we have the feasibility constraint that $\sum_{i=1}^n x_i \leq 1$. Thus, the social welfare is just the valuation of the winner, or 0 if there is no winner. An auction is welfare maximizing if, when bids are truthful, the auction outcome has the maximum possible social welfare.

Ideal Auctions (3)

Theorem 2.4 (Second-Price Auctions Are Ideal) A second price single-item auction satisfies the following:

- (1) [*strong incentive guarantees*] It is a DSIC auction.
- (2) [*strong performance guarantees*] It is welfare maximizing.
- (3) [*computational efficiency*] It can be implemented in time polynomial (indeed, linear) in the size of the input, meaning the number of bits necessary to represent the numbers $v_1, \dots, v_n$.

Case Study: Sponsored Search Auctions

A Web search results page comprises a list of organic search results—deemed relevant to your query by an algorithm like *PageRank*—and a list of sponsored links, which have been paid for by advertisers.

Every time you type a search query into a search engine, an auction is run in real time to decide which advertisers' links are shown, how these links are arranged visually, and what the advertisers are charged.

Around 2006, sponsored search auctions generated roughly 98% of Google's revenue.

The Basic Model of Sponsored Search Auctions (1)

The items for sale are k “slots” for sponsored links on a search results page. The bidders are the advertisers who have a standing bid on the keyword that was searched on. For example, **Volvo and Subaru** might be bidders on the keyword “station wagon,” while **Nikon and Canon** might be bidders on the keyword “camera.”

The Basic Model of Sponsored Search Auctions (2)

We quantify the difference between different slots using *click through rates* (CTR*s*). The CTR α_j of a slot j represents the probability that the end user clicks on this slot. Ordering the slots from top to bottom, we make the reasonable assumption that $\alpha_1 \geq \alpha_2 \geq \dots \geq \alpha_k$. For simplicity, we also make the unreasonable assumption that the CTR of a slot is independent of its occupant. Everything we'll say about sponsored search auctions extends to the more general and realistic model in which each advertiser i has a “quality score” β_i (the higher the better) and the CTR of advertiser i in slot j is the product $\beta_i \alpha_j$ (e.g., Exercise 3.4).

The Basic Model of Sponsored Search Auctions (3)

We assume that an advertiser is not interested in an impression (i.e., being displayed on a page) per se, but rather has a private valuation v_i for each click on her link. Hence, the expected value derived by advertiser i from slot j is $v_i \alpha_j$.

The Basic Model of Sponsored Search Auctions (4)

What We Want (desiderata):

- (1) **DSIC**. That is, truthful bidding should be a dominant strategy, and never leads to negative utility.
- (2) **Social welfare maximization**. That is, the assignment of bidders to slots should maximize $\sum_{i=1}^n x_i v_i \leq 1$, where x_i now denotes the CTR of the slot to which i is assigned (or 0 if i is not assigned to a slot). Each slot can only be assigned to one bidder, and each bidder gets only one slot.
- (3) **Computational efficiency**. The running time should be polynomial (or even near-linear) in the size of the input $v_1, \dots, v_n$. Remember that zillions of these auctions need to be run every day!

The Basic Model of Sponsored Search Auctions (5)

Our Design Approach

(1) the choice of who wins what, and (2) the choice of who pays what.

Step 1: Assume, without justification, that bidders bid truthfully. Then, how should we assign bidders to slots so that the above properties (2) and (3) hold?

Step 2: Given our answer to Step 1, how should we set selling prices so that the above property (1) holds?

The Basic Model of Sponsored Search Auctions (6)

Our Design Approach

(1) the choice of who wins what, and (2) the choice of who pays what.

Step 1: the natural greedy algorithm is optimal (and computationally efficient): for $i = 1, 2, \dots, k$, assign the i -th highest bidder to the i -th best slot.

Step 2: there is an analog of the second-price rule—sale prices that render truthful bidding a dominant strategy for every bidder. The next lecture gives an affirmative answer via **Myerson's lemma**, a powerful tool in mechanism design.

Section 3

Myerson's Lemma

Single-Parameter Environments

- Some number n of agents.
 - Each agent i has a private nonnegative valuation v_i
 - her value “per unit of stuff” that she acquires.
- There is a feasible set X .
 - Each element of X is a nonnegative n vector $(x_1, x_2, \dots, x_n)$
 - where x_i denotes the “amount of stuff” given to agent i .

Examples

Example 3.1 (Single-Item Auction) In a single-item auction (Section 2.1), X is the set of 0-1 vectors that have at most one 1—that is, $\sum_{i=1}^n x_i \leq 1$.

Example 3.2 (k-Unit Auction) With k identical items and the constraint that each bidder gets at most one (Exercise 2.3), the feasible set is the set of 0-1 vectors that satisfy $\sum_{i=1}^n x_i \leq k$.

Examples

Example 3.3 (Sponsored Search Auction) In a sponsored search auction (Section 2.6), X is the set of n -vectors corresponding to assignments of bidders to slots, where each slot is assigned to at most one bidder and each bidder is assigned to at most one slot. If bidder i is assigned to slot j , then the component x_i equals the click-through rate α_j of her slot.

Example 3.4 (Public Project) Deciding whether or not to build a public project that can be used by all, such as a new bridge, can be modeled by the set $X = \{(0, 0, \dots, 0), (1, 1, \dots, 1)\}$.

Allocation and Payment Rules

1. Collect bids $b = (b_1, \dots, b_n)$ from all agents. The vector b is called the bid vector or bid profile.
2. [**allocation rule**] Choose a feasible allocation $x(b) \in X \subseteq R^n$ as a function of the bids.
3. [**payment rule**] Choose payments $p(b) \in R^n$ as a function of the bids.

direct-revelation mechanisms vs

indirect mechanism (iterative ascending auction, exe. 2.7)

quasilinear utility model

In a mechanism with allocation and payment rules x and p
agent i receives utility

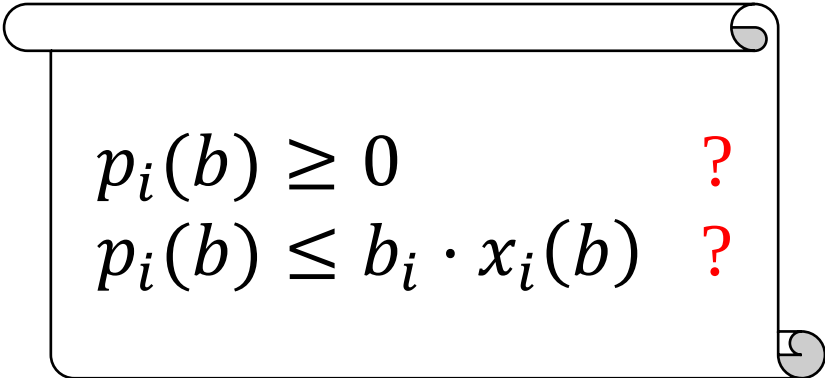
$$u_i(b) = v_i \cdot x_i(b) - p_i(b)$$

when the bid profile is b .

We focus on payment rules that satisfy

$$p_i(b) \in [0, b_i \cdot x_i(b)]$$

for every agent i and bid profile b .


$$p_i(b) \geq 0 \quad ?$$

$$p_i(b) \leq b_i \cdot x_i(b) \quad ?$$

Statement of Myerson's Lemma

Definition 3.5 (*Implementable Allocation Rule*) An allocation rule x for a single-parameter environment is implementable if there is a payment rule p such that the direct-revelation mechanism (x, p) is **DSIC**.

Definition 3.6 (*Monotone Allocation Rule*) An allocation rule x for a single-parameter environment is monotone if for every agent i and bids b_{-i} by the other agents, the allocation $x_i(z, b_{-i})$ to i is **nondecreasing** in her bid z .

Statement of Myerson's Lemma

Definition 3.5 (*Implementable Allocation Rule*) An allocation rule x for a single-parameter environment is implementable if there is a payment rule p such that the direct-revelation mechanism (x, p) is **DSIC**.

Definition 3.6 (*Monotone Allocation Rule*) An allocation rule x for a single-parameter environment is monotone if for every agent i and bids b_{-i} by the other agents, the allocation $x_i(z, b_{-i})$ to i is **nondecreasing** in her bid z .

bidding higher can only
get you more stuff.

Examples

For example, the **single-item auction allocation rule** that awards the item to the **highest bidder** is monotone: if you're the winner and you raise your bid (keeping other bids fixed), you continue to win.

By contrast, awarding the item to the **second-highest bidder** is a nonmonotone allocation rule: if you're the winner and you raise your bid high enough, you lose.

The **welfare-maximizing allocation rule** for sponsored search auctions (Example 3.3), with the **i -th highest bidder** awarded the **i -th best slot**, is monotone.

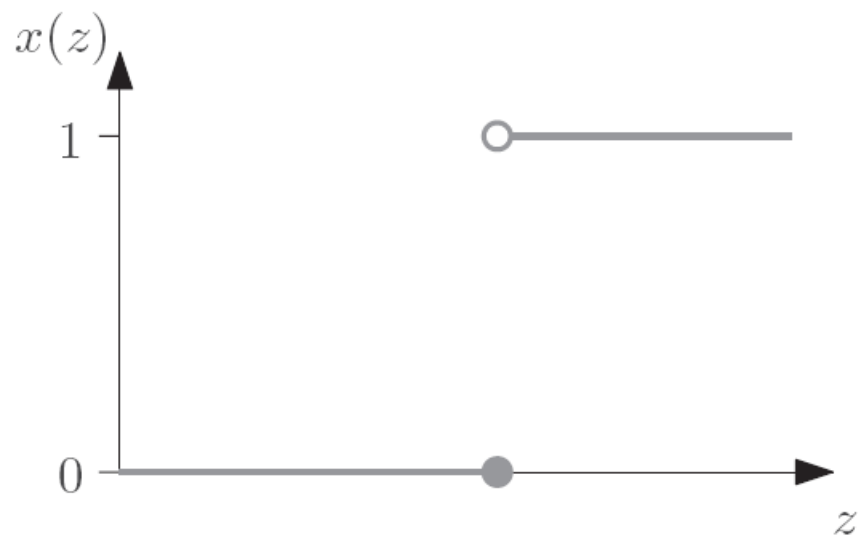
Myerson's Lemma

Theorem 3.7 (*Myerson's Lemma*) Fix a single-parameter environment.

- (a) An allocation rule x is **implementable** if and only if it is **monotone**.
- (b) If x is **monotone**, then there is a **unique payment rule** for which the direct-revelation mechanism (x, p) is **DSIC** and $p_i(b) = 0$ whenever $b_i = 0$.
- (c) The payment rule in (b) is given by **an explicit formula**.

Myerson's lemma is the foundation on which we'll build most of our mechanism design theory.

Myerson's Lemma



(a) 0-1 monotone curve



(b) Piecewise constant monotone curve

Figure 3.1: Examples of allocation curves $x(\cdot)$.

Proof by Picture

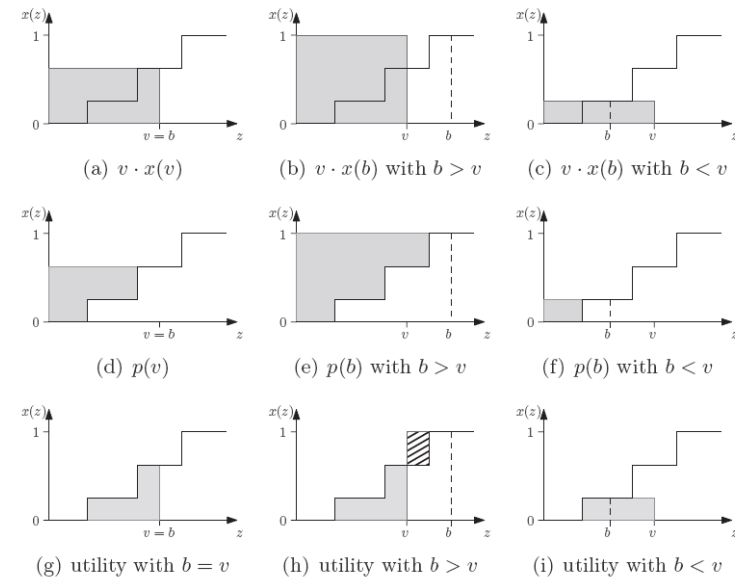


Figure 3.2: Proof by picture that the payment rule in (3.5), coupled with the given monotone and piecewise constant allocation rule, yields a DSIC mechanism. The three columns consider the cases of truthful bidding, overbidding, and underbidding, respectively. The three rows show the welfare $v \cdot x(b)$, the payment $p(b)$, and the utility $v \cdot x(b) - p(b)$, respectively. In (h), the solid region represents positive utility and the lined region represents negative utility.

Section 4

Dynamic Games and Learning

Section 5

Applications

Coordination in Software Development [midterm2011]

- There are n software developers, named $i = 1, \dots, n$.
- Each software developer i has an “*ideal specification*” θ_i for his product,
- but also would like his product to be compatible with the other software, where θ_i is a real number.
- Simultaneously, each i **selects** a specification parameter s_i , which is a real number.
- The payoff of a player i is

$$u_i(s_1, \dots, s_n) = 100 - (s_i - \theta_i)^2 - \frac{1}{n-1} \sum_{j \neq i} (s_i - s_j)^2$$

Note that a developers pays two costs: one for being away from his ideal specification, and one for being away from the specification of other softwares (cost of incompatibility).

Note also that in the normal-form game, the software developers are the players; each chooses a strategy s_i from real line, and the payoff function of player i is u_i .

Compute a Nash equilibrium

Solution:

For each player i the first-order condition for his best response is

$$2(s_i - \theta_i) - \frac{2}{n-1} \sum_{j \neq i} (s_i - s_j) = 0$$

which simplifies to

$$2s_i = \theta_i + \frac{1}{n-1} \sum_{j \neq i} s_j$$

To solve this equation system, sum it up over i and obtain $\sum_i s_i = \sum_i \theta_i$. Substituting this in the above equation, obtain

$$s_i = \frac{n-1}{2n-1} \theta_i + \frac{1}{2n-1} \sum_{j=1}^n \theta_j$$

Further Reading and Research

Projects

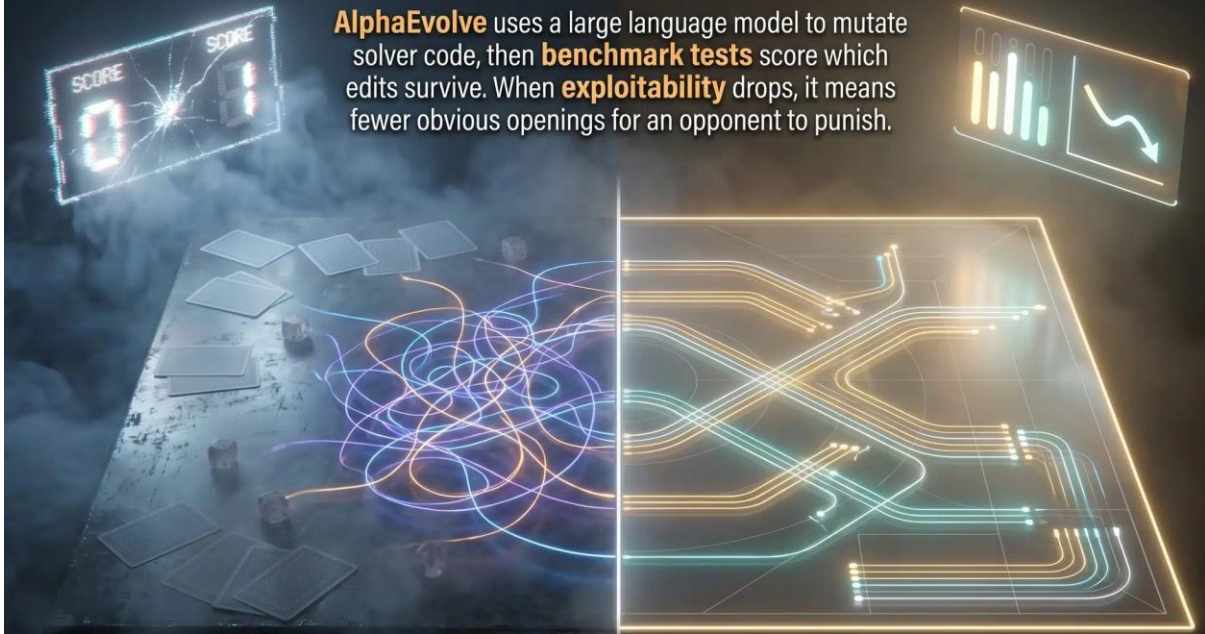
Startups

Resources

AlphaEvolve (Google Deep-mind)

STRATEGIC AI JUST STARTED UPGRADING THE RULES IT PLAYS BY

AlphaEvolve uses a large language model to mutate solver code, then **benchmark tests** score which edits survive. When **exploitability** drops, it means fewer obvious openings for an opponent to punish.



This is **game theory** moving from human tuning to evaluator-driven algorithm evolution in **multi-agent reinforcement learning**, built for **decision-making under uncertainty**.

(Image Credit: Intelligent Living)